Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. Search on a binary search tree in the worst case will do

a) n comparisons

b) n/2 comparisons

c) Log(n) to base 2 comparisons

d) Log(n) to base e comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. Search on a binary search tree in the worst case will do

a) n comparisons

b) n/2 comparisons

c) Log(n) to base 2 comparisons

d) Log(n) to base e comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The smallest key in a binary search tree will

a) Always be a leaf in the left subtree

b) Always be a leaf in the right subtree

c) Could be an internal node with no left subtree

d) Could be an internal node with no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The smallest key in a binary search tree will

a) Always be a leaf in the left subtree

b) Always be a leaf in the right subtree

c) Could be an internal node with no left subtree

d) Could be an internal node with no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. A binary tree has $n$nodes. What is the maximum possible number of nodes at level k, assuming the root is at level 0?

a) k
b) $2k$
c) 2^k
d) 2^{k+1}

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. A binary tree has $n$nodes. What is the maximum possible number of nodes at level k, assuming the root is at level 0?

a) k
b) 2k
c) 2^k
d) 2^{k+1}

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. A binary tree contains 15 nodes. What is the minimum possible height of the tree, where the height of a single-node tree is 0?

a) 2
b) 3
c) 4
d) 14

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. A binary tree contains 15 nodes. What is the minimum possible height of the tree, where the height of a single-node tree is 0?

a) 2
b) 3
c) 4
d) 14

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. Which statement is always true for a binary tree?

a) Every node has exactly two children
b) Every leaf has two children
c) A node can have at most two children
d) Every internal node has two children

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. Which statement is always true for a binary tree?

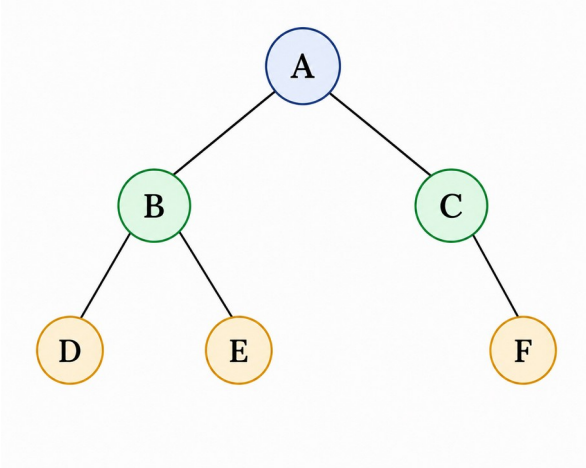
a) Every node has exactly two children
b) Every leaf has two children
c) A node can have at most two children
d) Every internal node has two children

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Definitiion

- A **binary tree** is a tree in which every node has **at most two children**.
- The two children are distinguished as the **left child** and **right child**.
- A binary tree is either:
  - An **empty tree**, or
  - A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**



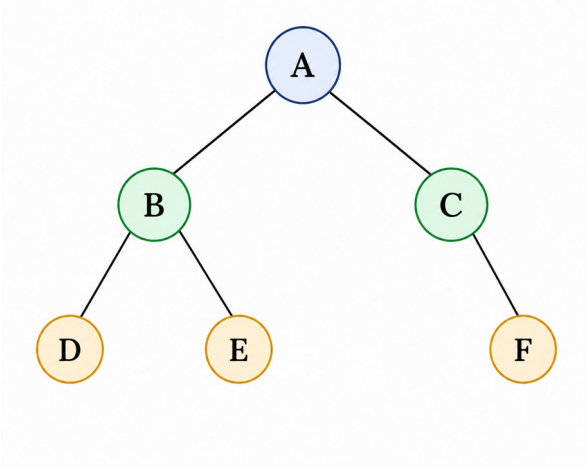
© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Inorder traversal of the binary tree

- If tree is not empty
  - Visit the left subtree in inorder
  - Visit the root
  - Visit the right subtree in inorder

Inorder traversal of this tree:
D B E A C F



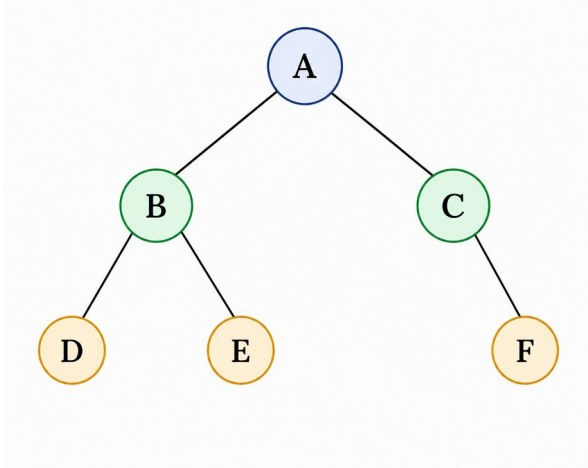
© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

preorder traversal of the binary tree

- If tree is not empty
  - Visit the root
  - Visit the left subtree in preorder
  - Visit the right subtree in preorder

preorder traversal of this tree:
A B D E C F

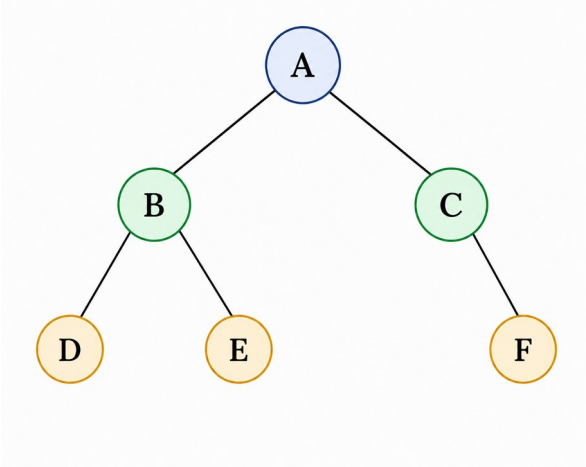


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

postorder traversal of the binary tree

- If tree is not empty
  - Visit the left subtree in postorder
  - Visit the right subtree in postorder
  - Visit the root



postorder traversal of this tree:
D E B F C A

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree

- Every node has a key
- Nodes are inserted based on the key
- If the key of a node is less than that of the root, it will be inserted in the left subtree
- Otherwise, it will be inserted in the right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary Search Tree : simple and small interface

- Create a tree
- Destroy the tree
- Insert the key
- Display the tree in inorder

- Find the number of nodes
- Find the number of leaves

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : structure of the functions

- Recursive on nodes and not on tree
- Model for recursive routine :
  - Empty tree
  - Tree with left and right subtrees

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary Search Tree : interface

```
void init(tree_t *ptr_tree);
        Ground the root.
void deinit(tree_t *ptr_tree);
        Walk through the tree
        Remove the left subtree
        Remove the right subtree
        Remove the root
void disp(tree_t *ptr_tree);
       // This is inorder traversal of the tree
        If the tree is not empty
                Display the left subtree
                Display the root
                Display the right subtree
```

```
void insert(tree_t *ptr_tree, int key);
        If the tree is empty, make this the
        root
        Otherwise compare the keys
        Decide to insert in the left subtree
        or the right subtree
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : insert

```
// on the tree; non recursive
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : insert

```
// on the node; recursive; observe the assignment to the root
static node_t* insert_recursive(node_t* root, node_t* temp)
{
        if(root == NULL)
        {
                root = temp;
        }
        else if(root->key_ > temp->key_)
        {
                root->left_ = insert_recursive(root->left_, temp);
        }
        else
        {
                root->right_ = insert_recursive(root->right_, temp);
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

1. Count the number of nodes

```
int number_of_nodes_recursive(node_t* temp)
{
        if(temp == NULL)
        {
                return 0;
        }
        else
        {
                return 1 + number_of_nodes_recursive(temp->left_)
                        + number_of_nodes_recursive(temp->right_);
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

2. Count the number of leaves

```
int number_of_leaves_recursive(node_t* temp)
{
        if(temp == NULL)
        {
                return 0;
        }
        else if(temp->left_ == NULL && temp->right_ == NULL)
        {
                return 1;
        }
        else
        {
                return number_of_leaves_recursive(temp->left_)
                        + number_of_leaves_recursive(temp->right_);
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

3. Compare two trees

```
int are_same_recursive(node_t* root1, node_t* root2)
{
        if(root1 == NULL && root2 == NULL)
                return 1;
        else if(root1 == NULL || root2 == NULL)
                return 0;
        else
                return root1->key_ == root2->key_ &&
                        are_same_recursive(root1->left_, root2->left_) &&
                        are_same_recursive(root1->right_, root2->right_);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

4. Copy tree

```
node_t* copy_recursive(node_t* ptr_src)
{
        node_t *temp;
        if(ptr_src != NULL)
        {
                temp = (node_t*)malloc(sizeof(node_t));
                temp->key_ = ptr_src->key_;
                temp->left_ = copy_recursive(ptr_src->left_);
                temp->right_ = copy_recursive(ptr_src->right_);
                return temp;
        }
        else
        {
                return NULL;
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

5. search

```
node_t* search_recursive(node_t* root, int key)
{
        if(root == NULL)
                return NULL;
        else if(key == root->key_)
                return root;
        else if(key < root->key_)
                return search_recursive(root->left_, key);
        else
                return search_recursive(root->right_, key);

}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

6. Inorder successor if the argument is the right subtree of a given node

```
node_t* inorder_successor(node_t* root)
{
        node_t* temp = root;
        while(temp->left_ != NULL)
                temp = temp->left_;
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key

```
node_t* delete_key_recursive(node_t* root, int key)
{
    if(root == NULL) // empty tree
   {
                return NULL;
   }
   else if(key < root->key_) // search in left subtree
   {
      root->left_ = delete_key_recursive(root->left_, key);
   }
   else if (key > root->key_) // search in right subtree
   {
      root->right_ = delete_key_recursive(root->right_, key);
   }
  else  // match found; action of deletion should start
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key continued

```
if(root->left_ == NULL && root->right_ == NULL) // 1 leaf
{
        free(root);    return NULL;
}
else if(root->left_ == NULL) // 2. no left
{
        node_t* temp = root->right_; free(root);
        return temp;
}
else if(root->right_ == NULL) // 3. no right
{
        node_t* temp = root->left_;  free(root);
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key continued; we have found a match

```
                // 4. has both left and right
                // find the inorder successor
                else
                {
                        node_t* temp = inorder_successor(root->right_);
                        root->key_ = temp->key_; // replace
                        // search with the new key in the right subtree
                        root->right_ = delete_key_recursive(root->right_, temp->key_);
                }
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : persistence

- A single traversal will not define the tree uniquely
- An inorder and a postorder traversal combined define a tree uniquely
- Rightmost node in the postorder traversal will be the root of the tree
- This node in the order traversal divides the tree in left subtree and the right subtree
- Repeating traversal from right to left of post order traversal and locating the node
  to divide the inorder traversal to left and right subtrees
  will recreate the tree

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

1. In 0 based indexing , mapping the root i to its left child is

a) trunc(i/2)

b) 2*i

c) 2*i+1

d) Round(i/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

1. In 0 based indexing , mapping the root i to its left child is

a) trunc(i/2)

b) 2*i

c) 2*i+1

d) Round(i/2)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

2. In node based array representation of a tree, a leaf is identified when its

a) left = -1 and right = -1

b) left = 0 and right = 0

c) Left = -1 is sufficient

d) Left = 0 is sufficient

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

2. In node based array representation of a tree, a leaf is identified when its

a) left = -1 and right = -1

b) left = 0 and right = 0

c) Left = -1 is sufficient

d) Left = 0 is sufficient

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

3. In 1 based indexing , the highest index in the array representation of a complete binary tree of n nodes is

a) n

b) n - 1

c) Log(n, 2)

d) n + 1

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

3. In 1 based indexing , the highest index in the array representation of a complete binary tree of n nodes is

a) n

b) n - 1

c) Log(n, 2)

d) n + 1

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

4. In 1 based indexing , the leaf node i in the array representation of a complete binary tree of n nodes will satisfy the following property

a) i > n

b) 2i > n

c) 2i >= n

d) i > 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

4. In 1 based indexing , the leaf node i in the array representation of a complete binary tree of n nodes will satisfy the following property

a) i > n

b) 2i > n

c) 2i >= n

d) i > 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

5. In 1 based indexing , a tree of n nodes which is not a complete tree may have a node stored at the highest index

a) n

b) 2^n -1

c) 2*n

d) 2^n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree implemented as an array

5. In 1 based indexing , a tree of n nodes which is not a complete tree may have a node stored at the highest index

a) n

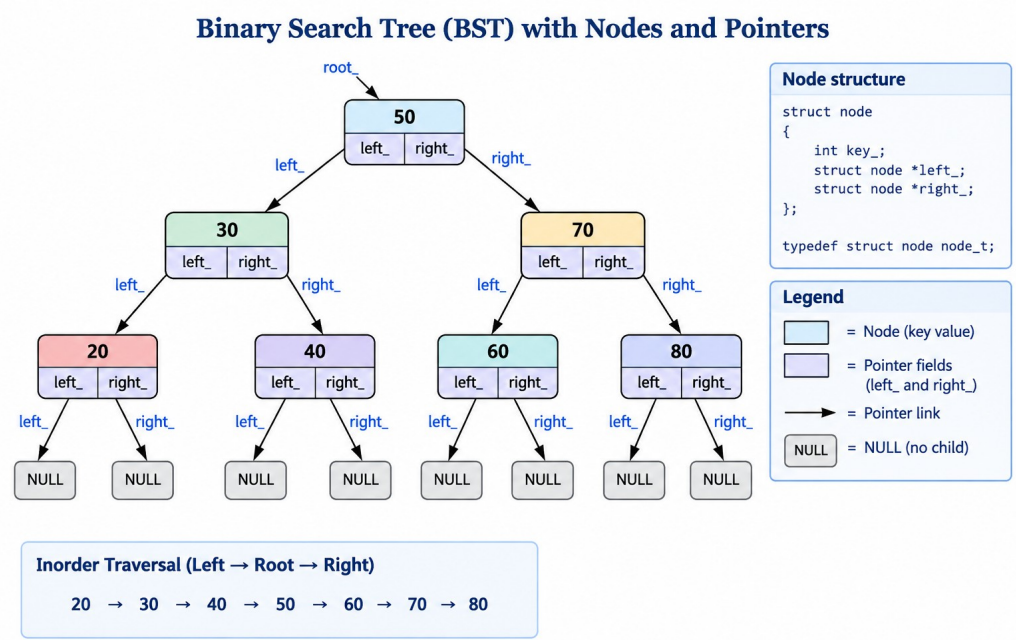
b) 2^n -1

c) 2*n

d) 2^n

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation



© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

1. Based on relative position of parent and children

a)

0 index based mapping:

Parent : i , left child : 2i + 1; right child : 2i + 2

b)

1 index based mapping :

Parent : i , left child : 2i ; right child : 2i + 1

index	key
0	
1	50
2	30
3	70
4	20
5	40
6	60
7	80

Points:

- Compact representation if the tree is a complete tree
- If not complete, we should know which slots are not filled
- Might require a dummy value

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

- Each node has element, left and right
- Left and right are indices

Points:

- Require a memory manager to allocate and deallocate a free slot
- Otherwise insertion limited by the size of the array and not on available slots free on deletion

index	left	key	right
0	1	50	2
1	3	30	4
2	5	70	6
3	-1	20	-1
4	-1	40	-1
5	-1	60	-1
6	-1	80	-1

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

- Structure

```
#define MAXSIZE 200
struct node
{
        int key_;
        int left_;
        int right_;
};
typedef struct node node_t;
```

```
struct tree
{
        int root_;
        int count_;
        node_t elem_[MAXSIZE];
};
typedef struct tree tree_t;
```

- Count is used to indicate the next slot to insert as there is memory manager

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure
Interface: no change

```
void init(tree_t *ptr_tree);
void deinit(tree_t *ptr_tree);
void disp(tree_t *ptr_tree);
void insert(tree_t *ptr_tree, int key);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Initialize :

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = -1; // index of the first element; invalid to start
        ptr_tree->count_ = 0; // # of elem or slot to insert
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

insert :

```
void insert(tree_t *ptr_tree, int key)
{
// no space; exit
if(ptr_tree->count_  == MAXSIZE)
{
        printf("tree full\n");
        exit(1);
}
// count : empty slot at which key, left and right
// can be populated
// increment count to indicate the number of elem
// new node is always a leaf
int temp = ptr_tree->count_++;
ptr_tree->elem_[temp].key_ = key;
ptr_tree->elem_[temp].left_ =
        ptr_tree->elem_[temp].right_ = -1;
// call recursive to make the right connections
ptr_tree->root_ = insert_recursive(ptr_tree->elem_,
        ptr_tree->root_, temp);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Recursive insert :

```
static int insert_recursive(node_t elem[], int root, int temp)
{
        if(root == -1)
        {
                root = temp;
        }
        else if(elem[root].key_ > elem[temp].key_)
        {
                elem[root].left_ = insert_recursive(elem, elem[root].left_, temp);
        }
        else
        {
                elem[root].right_ = insert_recursive(elem, elem[root].right_, temp);
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Display:

```
void disp(tree_t *ptr_tree)
{
    printf("Tree : ");
    disp_recursive(ptr_tree->elem_, ptr_tree->root_);
    printf("\n");
}
```

```
static void disp_recursive(node_t elem[], int temp)
{
    if(temp != -1)
    {
            disp_recursive(elem, elem[temp].left_);
            printf("%d ", elem[temp].key_);
            disp_recursive(elem, elem[temp].right_);
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

```
# of nodes
int number_of_nodes(tree_t *ptr_tree)
{
  return   number_of_nodes_recursive(
ptr_tree->elem_,
              ptr_tree->root_);
}
```

```
int number_of_nodes_recursive(node_t elem[], int
temp)
{
   if(temp == -1)
   {
        return 0;
   }
   else
   {
        return 1 +
number_of_nodes_recursive(elem,
elem[temp].left_)       +
number_of_nodes_recursive(elem,
elem[temp].right_);
                }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

```
# of leaves
int number_of_leaves(tree_t *ptr_tree)
{
   return number_of_leaves_recursive
                (ptr_tree->elem_,
                ptr_tree->root_);
}
int number_of_leaves_recursive(node_t elem[], int temp)
{
                if(temp == -1)
                {
                                return 0;
                }
else if(elem[temp].left_ == -1 &&       elem[temp].right_ == -1)
{
                return 1;
}
else
{
    return number_of_leaves_recursive(
        elem, elem[temp].left_) +
number_of_leaves_recursive(
                elem, elem[temp].right_);
}
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : Array representation

2. Array implementation of binary tree with the node structure

Other functions:

- Traversals :
  - Inorder
  - Preorder
  - Postorder
- Compare two trees
- Copy tree

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

1. The number of pointers grounded in a tree of n nodes is

a) n + 1

b) n - 1

c) n

d) 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

1. The number of pointers grounded in a tree of n nodes is

a) n + 1

b) n - 1

c) n

d) 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

2. Left of new node inserted in a threaded binary search tree(for inorder traversal)

a) Is always a structural pointer

b) Is a thread and gets the value of left of parent

c) Is a thread and gets the value of right of parent

d) Is always grounded

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

2. Left of new node inserted in a threaded binary search tree(for inorder traversal)

a) Is always a structural pointer

b) Is a thread and gets the value of left of parent

c) Is a thread and gets the value of right of parent

d) Is always grounded

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

3. Lthread = 1 implies that the left is a thread. Rthread = 1 implies that the right is a thread. Initializing lthread and rthread of a new node

a) to 0 is preferred as left and right pointers of a new node are always structural

b) to 1 is preferred as left and right pointers of a new node are always threads

c) to 0 is preferred as left and right pointers of a new node are always threads

d) to 1 is preferred as left and right pointers of a new node are always structural

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

3. Lthread = 1 implies that the left is a thread. Rthread = 1 implies that the right is a thread. Initializing lthread and rthread of a new node

a) to 0 is preferred as left and right pointers of a new node are always structural

b) to 1 is preferred as left and right pointers of a new node are always threads

c) to 0 is preferred as left and right pointers of a new node are always threads

d) to 1 is preferred as left and right pointers of a new node are always structural

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

4. Lthread = 1 and Rthread = 1 implies that

a) the node is a leaf

b) the node is an internal node

c) The node should be the root

d) Nothing can be said about the node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

4. Lthread = 1 and Rthread = 1 implies that

a) the node is a leaf

b) the node is an internal node

c) The node should be the root

d) Nothing can be said about the node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

5. When a new is added, the right of the node

a) Is a thread and refers to its parent

b) Is structural and refers to its parent

c) Is a thread and refers to itself

d) Is structural and refers to itself

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

5. When a new is added, the right of the node

a) Is a thread and refers to its parent

b) Is structural and refers to its parent

c) Is a thread and refers to itself

d) Is structural and refers to itself

© Confidential Information of PES University. All Rights Reserved - 2026

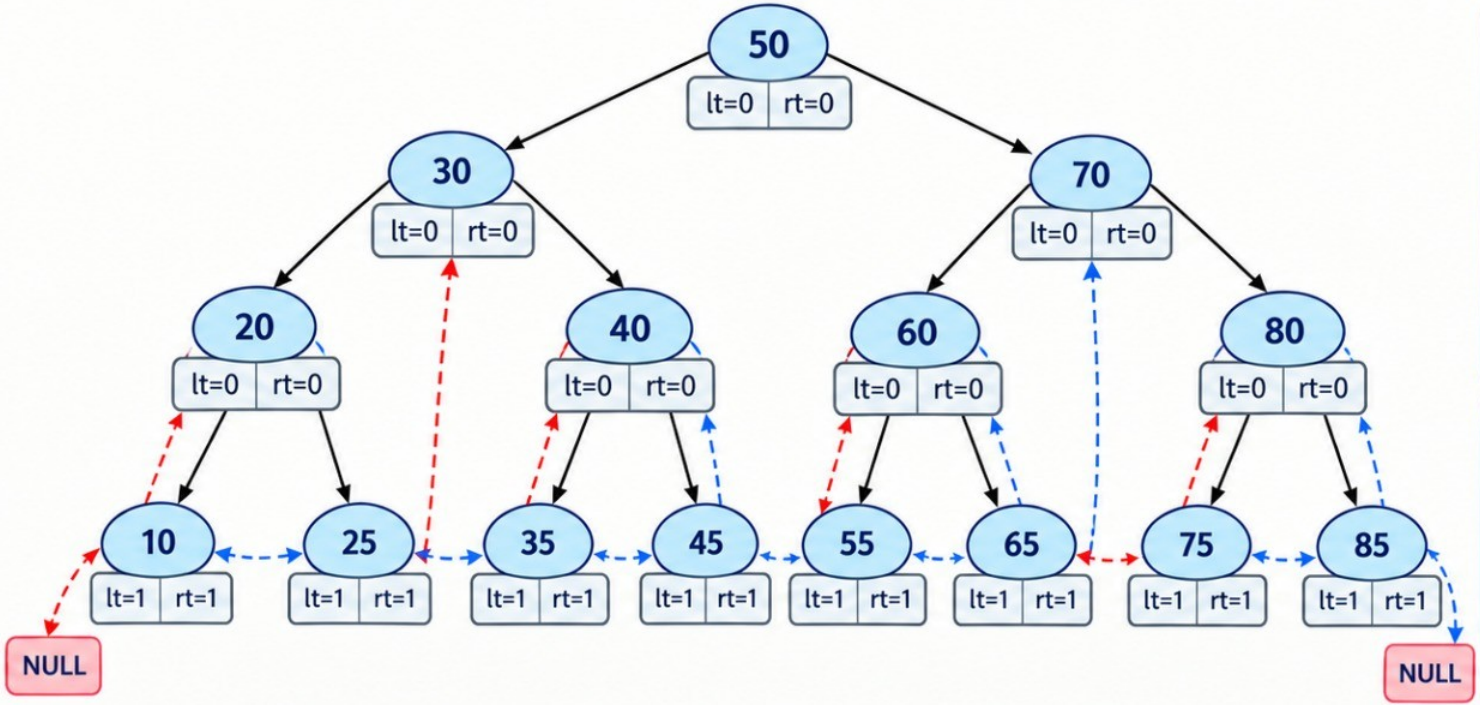
Threaded Binary Tree

- In a binary tree of n nodes, there are 2n pointers
- A tree has n-1 edges.
- n + 1 pointers are grounded

- Make the right pointers which are normally grounded to point the successor in some traversal
- Make the left pointers which are normally grounded to point the predecessor in some traversal
- Avoid recursive calls while traversing – so efficient in time
- Requires extra space to indicate whether a pointer is a structural pointer or a thread
- Inorder traversal selected in our discussion

© Confidential Information of PES University. All Rights Reserved - 2026

Fully Threaded Binary Tree (with Inorder Threads)



Legend

= Child link

= Left thread (Inorder predecessor)

= Right thread (Inorder successor)

key | lt | rt = Node structure (lt = 0/1, rt = 0/1)

Node structure (in code)

```
typedef struct node {
    int key;
    struct node *left;
    struct node *right;
    int lthread;
    int rthread;
}
typedef struct node node_t;
```

Inorder sequence (via threads)

NULL ↔ 10 → 20 → 25 → 30 → 35 → 40 → 45 → 50 → 55 → 60 → 65 → 70 → 75 → 80 → 85 → NULL

(Threads connect nodes in inorder sequence)

Threaded Binary Tree

- Structure:

```
struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
        int lthread_; // 1 if left_ is a thread
        int rthread_; // 1 if right_ is a thread
};
typedef struct node node_t;
```

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- interface:

```
void init(tree_t *ptr_tree);
void deinit(tree_t *ptr_tree);
void disp(tree_t *ptr_tree);
void insert(tree_t *ptr_tree, int key);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- initialize:

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = NULL;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- insert: helper function

```
static node_t* create_node(int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("no more memory\n"); exit(1);
        }
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL; // should be changed on insertion
        temp->lthread_ = temp->rthread_ = 1; // should be threads
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert:
Insert_key is not available to the client
This knows the node structure.

```
void insert(tree_t *ptr_tree, int key)
{
        ptr_tree->root_ = insert_key(ptr_tree->root_, key);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert:

```
static node_t* insert_key(node_t* root, int key)
{
   node_t *temp = create_node(key);
   node_t *parent = NULL;
   node_t *curr = root;
   while (curr != NULL)
   {
      parent = curr;
      if (key < curr->key_)
      {
         if (curr->lthread_ == 0)
                curr = curr->left_;
         else
                 break;
      }
```

```
      else
      {
            if (curr->rthread_ == 0)
                  curr = curr->right_;
            else
                  break;
      }
   } // of while
```

if key is less than that of the root move left until there is left subtree. Lthread = 1 => no left subtree

If key is not less than that of the root move right until There is no right subtree. Rthread = 1 => no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert: continued

```
if (parent == NULL)
  {
     return temp; // empty tree case
  }
   /* Insert as left child */
  if (key < parent->key_)
  {
     temp->left_ = parent->left_;
     temp->right_ = parent;

     parent->left_ = temp;
     parent->lthread_ = 0;
  }
```

temp->right_ is a thread and refer to its parent. parent->left_ refers to temp and is no more a thread.

```
 /* Insert as right child */
   else
   {
      temp->right_ = parent->right_;
      temp->left_ = parent;

      parent->right_ = temp;
      parent->rthread_ = 0;
   }

   return root;
}
```

Insertion on the right subtree is similar.

© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

Inorder traversal :

```
 void disp(tree_t *ptr_tree)
{
                printf("Tree : ");
                inorder(ptr_tree->root_);
                printf("\n");
}
```

Helper function:

```
// returns the node with no left subtree
static node_t *leftmost(node_t *temp)
{
   if (temp == NULL)
      return NULL;

   while (temp->lthread_ == 0)
      temp = temp->left_;

   return temp;
}
```

```
static void inorder(node_t *root)
{
  // start from the leftmost node in the tree
   node_t *temp = leftmost(root);

   while (temp != NULL)
   {
      printf("%d ", temp->key_);
     // find the inorder successor
    // follow the right thread if the pointer is a thread
      if (temp->rthread_ == 1)
         temp = temp->right_;
      else
     // otherwise find the leftmost of its right subtree
         temp = leftmost(temp->right_);
   }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

- An expression tree is a tree where the internal nodes are operators and leaves are operands
- **Algorithm**:
  - **Input** : an expression in postfix. For simplicity, the operands are assumed to be single digits.
  - **Process** : construct an expression tree
  - **Output**: find the value of the expression

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Phase 1:
Construct the expression tree:

Data structure used: stack of tree nodes

- Scan the input left to right
- If operand, create a tree node and push it to a stack of tree nodes.
- If operator, pop two tree nodes from the stack, create a new tree node with the operator as the key.
- Attach the popped nodes as subtrees of the new node.
- Push the new node into the stack.

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Phase 2 :

- Evaluate the expression tree:
- Stack should have one and only one tree node.

- On this tree, do the evaluation.
- If the node is a leaf,
  - return the operand.
- Else
  - Evaluate the left subtree
  - Evaluate the right subtree
  - Apply the operator of this node on these two results

© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

Structure:

```
#define MAXSIZE 100
struct node
{
        char ch_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;

struct stack
{
        node_t* key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

interface:

```
void init_stack(stack_t *ptr_stack);
void deinit_stack();
// push modified to take care of operand and operator differently
void push_operand(stack_t *ptr_stack, char ch);
void push_operator(stack_t *ptr_stack, char ch);
// pop returns a pointer to a tree node
node_t* pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:
create_node: helper function

```
static node_t* create_node(char ch)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("memory full\n"); exit(1);
        }
        temp->ch_ = ch;
        temp->left_ = temp->right_ = NULL;
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

push_operand: single char operand; could use a union of operator and operand

```
void push_operand(stack_t *ptr_stack, char ch)
{
        if(! is_full(ptr_stack))
        {
                node_t* temp = create_node(ch);
                ptr_stack->key_[++ptr_stack->top_] = temp;
        }
        else
        {
                printf("stack full; cannot push\n");
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

```
void push_operator(stack_t *ptr_stack, char ch)
{
        if(! is_full(ptr_stack))
        {
                node_t* temp = create_node(ch);
                temp->right_ = pop(ptr_stack);
                temp->left_ = pop(ptr_stack);
                ptr_stack->key_[++ptr_stack->top_] = temp;
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation:

```
node_t* pop(stack_t *ptr_stack)
{
        if(! is_empty(ptr_stack))
        {
                return ptr_stack->key_[ptr_stack->top_--];
        }
        else
        {
                printf("stack empty; cannot pop\n");
                return '\0';
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Expression Tree

implementation: stack should have one element - root of the expression tree.

- Pass this as argument to eval_recursive.
- If leaf, then its an operand – find and return the numeric value of the character.
- Evaluate left then evaluate right
- Based on the operator, combine the two results.

```
int eval_recursive(node_t* root)
{
   int left, right;
   if (root->left_ == NULL &&
                  root->right_ == NULL)
      return root->ch_ - '0';

   left = eval_recursive(root->left_);
   right = eval_recursive(root->right_);
```

```
switch (root->ch_)
   {
      case '+': return left + right;
      case '-': return left - right;
      case '*': return left * right;
      case '/': return left / right;
   }
    return 0;
}
 int eval(stack_t *ptr_stack)
{
return eval_recursive(pop(ptr_stack));
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

1. Searching for a key in a maxheap in the worst case makes

a) n comparisons

b) n – 1 comparisons

c) floor(n/2) comparisons

d) ceil(n/2) comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

1. Searching for a key in a maxheap in the worst case makes

a) n comparisons

b) n – 1 comparisons

c) floor(n/2) comparisons

d) ceil(n/2) comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

2. adding a key in a maxheap in the worst case makes

a) n operations

b) n – 1 operations

c) floor(n/2) operations

d) ceil(n/2) operations

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

2. adding a key in a maxheap in the worst case makes

a) n operations

b) n – 1 operations

c) floor(n/2) operations

d) ceil(n/2) operations

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

3. Finding the maximum key in a maxheap in the worst case makes

a) n operations

b) n – 1 operations

c) 1 operation

d) n/2 operations

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

3. Finding the maximum key in a maxheap in the worst case makes

a) n operations

b) n – 1 operations

c) 1 operation

d) n/2 operations

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

4. What happens if the left subtree and the right subtree at each node are swapped?

a) The tree is no more a heap

b) The max heap becomes a heap tree

c) The max heap remains to be a max heap

d) Depends on the keys in the tree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

4. What happens if the left subtree and the right subtree at each node are swapped?

a) The tree is no more a heap

b) The max heap becomes a heap tree

c) The max heap remains to be a max heap

d) Depends on the keys in the tree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

5. A heap has n elements with its root at index 1. Which condition will tell us that the ith node does not have a right subtree?

a) 2i + 1 > n

b) 2i > n

c) 2i + 1 >= n

d) 2i >= n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Heap

5. A heap has n elements with its root at index 1. Which condition will tell us that the ith node does not have a right subtree?

a) 2i + 1 > n

b) 2i > n

c) 2i + 1 >= n

d) 2i >= n

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

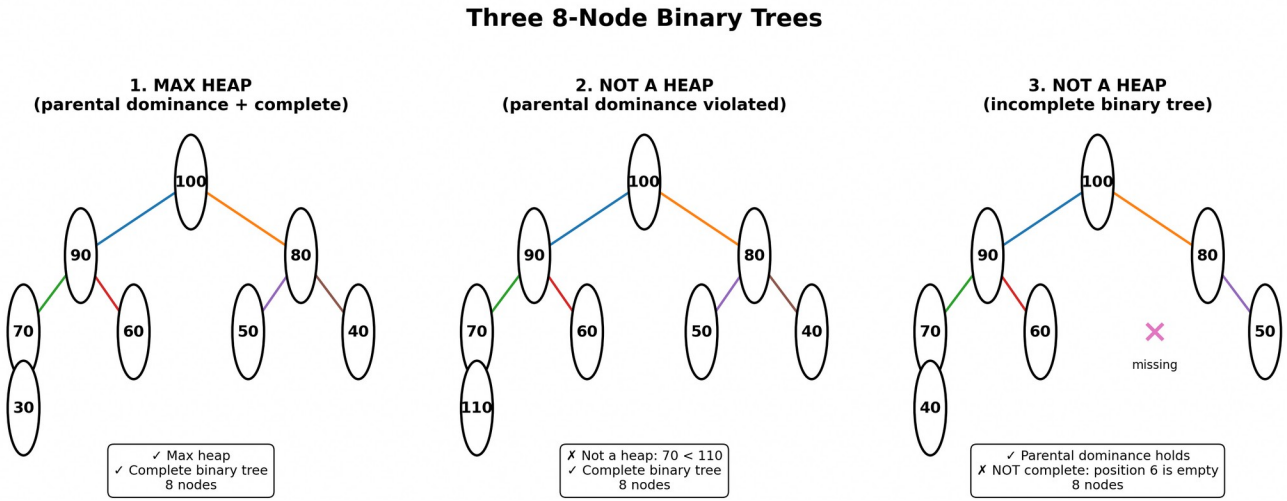
A heap(maxheap) is a tree with two special properties.

1. Complete binary tree:
   The nodes are filled up at each level.
   At the bottom most level, leftmost nodes are filled up.

2. Parental dominance:
   The key at the root is greater than the keys of its subtrees recursively.

© Confidential Information of PES University. All Rights Reserved - 2026

Heap



© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Heap Implementation:

- Always as an array
- Method 1:
    - Start from index 0
    - Leftchild(i) = 2i + 1
    - Rightchild(i) = 2i + 2
    - Parent(i) = floor((i-1)/2)
- Method 2:
    - Start from index 1
    - Leftchild(i) = 2i
    - Rightchild(i) = 2i + 1
    - Parent(i) = floor(i/2)
- We will use the method 2

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Advantages of heap:

- No pointers
- No extra storage
- Easy way to determine empty or full
- Simple way to traverse either as an array or as a tree
- Does not waste memory as the tree is a complete binary tree

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

uses of heap:

- Sorting : not commonly used
- Finding max(or min) repeatedly
  - Example: priority queue
  - Example: n way merge

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Conversion of a given array to a heap

- Refer heap1 folder; heap.h, heap.c and client.c
- Heap structure:

```
struct heap
{
        int key_[MAXSIZE];
        int n_;
};
typedef struct heap  heap_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Conversion of a given array to a heap

- Copy the given array to the heap
- Leaves are always heaps
- From the first internal node, till the root, move indices backwards
    - Make each subtree a heap(heapify)

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Conversion of a given array to a heap

- Heapify:
- Make the heap of n nodes heapify
  about the ith node

```
static void heapify(int a[], int n, int pos)
{
        int i = pos;
        int j = 2 * i;
        int val = a[i];
        int is_heap = 0;
        while(j <= n && ! is_heap){
                if(j + 1 <= n && a[j+1] > a[j])
                        ++j;
                if(a[j] > val) {
                        a[i] = a[j];
                        i = j;
                        j = 2 * i;
                }
                else {
                        is_heap = 1;
                }
        }
a[i] = val;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement insert a node at the end and remove the maximum

- Refer the folder heap2 – heap.h heap.c
- void insert(heap_t *ptr_heap, int key);
- int delete_max(heap_t *ptr_heap);
-

© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement insert a node at the end and remove the maximum

- Refer the folder heap2 – heap.h heap.c
- Insert : add at the end; check whether the parent tree is a heap repeatedly until the root is reached

```
void insert(heap_t *ptr_heap, int key)
{
        int pos = ++ptr_heap->n_;
                ptr_heap->key_[pos] = key;
        while(pos > 1 && ptr_heap->key_[pos] > ptr_heap->key_[pos/2])
        {
                swap(&ptr_heap->key_[pos], &ptr_heap->key_[pos/2]);
                pos = pos / 2;
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement insert a node at the end and remove the maximum

- Refer the folder heap2 – heap.h heap.c
- Delete max : replace the first node with the last and heapify about the root.

```
int delete_max(heap_t *ptr_heap)
{
        int res = ptr_heap->key_[1];
        ptr_heap->key_[1] = ptr_heap->key_[ptr_heap->n_];
        heapify(ptr_heap->key_, --ptr_heap->n_, 1);
        return res;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement a priority queue using heap

- Refer the folder priority queue –queue.h queue.c heap.h heap.c
- The queue data structure embeds the heap. The functions of the priority queue like enqueue and dequeue are forwarded to the embedded heap.

```
struct node
{
        int key_;
        struct node* next_;
};
typedef struct node node_t;
 struct queue
{
        heap_t q_;
};
typedef struct queue queue_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement a priority queue using heap

```
void enqueue(queue_t *ptr_queue, int key)
{
        insert(&ptr_queue->q_, key);

}
int dequeue(queue_t *ptr_queue)
{
        return delete_max(&ptr_queue->q_);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Heap

Implement a priority queue using heap

- In other implementations of priority queue, one of the operations(enqueue or dequeue) would not be that efficient – the number of operations would depend on the number of elements in the queue
- In this implementation, both the operations are proportional to log(n, 2) where is the number of elements

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree

AVL tree: height balanced binary search tree

- Searching in a binary search tree may require n comparisons
- The tree could become skewed – degenerate to a list
- AVL tree gives a solution to this problem
- Each node has a balance factor
  - Height(left subtree) – Height(right subtree)
- Difference is maintained within 1 (-1, 0, 1)
- Rotate the tree to keep the balance

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: rotations

Case	Configuration	Rotation
LL	Left-Left	Right rotation
RR	Right-Right	Left rotation
LR	Left-Right	Left rotation + Right rotation
RL	Right-Left	Right rotation + Left rotation

Same direction → single rotation
Opposite directions → double rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LL Rotation**

Example

Insert: 30, 20, 10

The resulting BST is:

```
      30
     /
    20
   /
  10
```

The path from the unbalanced node is:

30 → 20 → 10

Both moves are **left**. Therefore, this is an **LL case**.

Balance factor

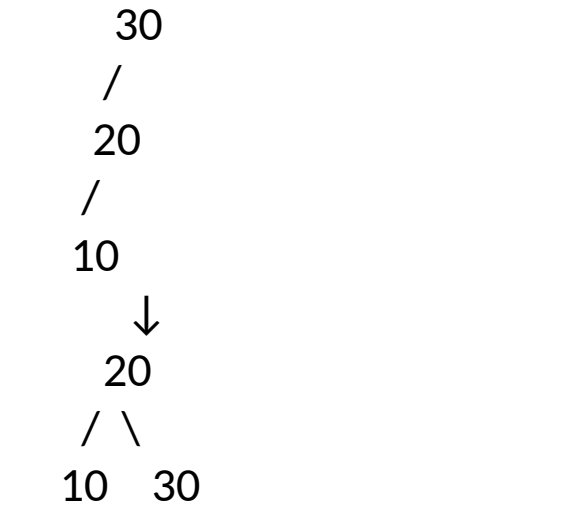
At node 30: Left height = 2; Right height = 0; Therefore: $BF(30) = 2$

The tree is left-heavy

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LL Rotation**

Solution: Right Rotation : Rotate right around 30:



The tree is now balanced.

LL rule : LL → Right Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RR Rotation**

Now consider inserting: 10, 20, 30

The BST becomes:

```
  10
    \
     20
       \
        30
```

The path is:

$10 \rightarrow 20 \rightarrow 30$

Both moves are **right**. Therefore, this is an **RR case**.

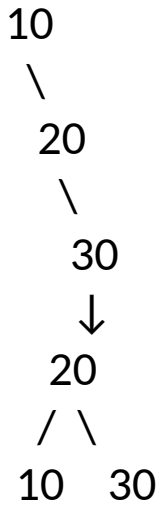
At node 10:

$BF(10) = 0 - 2 = -2$ The node is right-heavy.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RR Rotation**

Solution: Solution: Left Rotation; Rotate left around 10:



RR rule

RR → Left Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Insert: 30, 10, 20

The BST becomes:

```
    30
   /
  10
   \
    20
```

The path is:

30 → 10 → 20

The first move is **left**, but the second is **right**. Therefore:

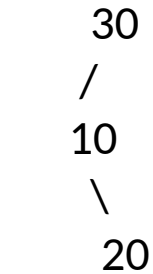
Left → Right

This is an **LR case**. (requires double rotation)

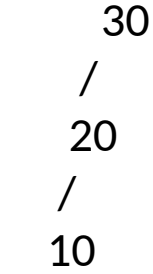
© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Step 1 : **Left rotation :** First rotate the subtree rooted at 10 to the left.



becomes:

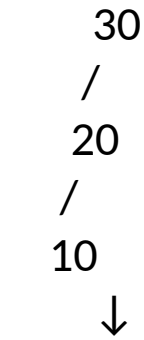


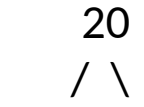
Now the tree has become an LL configuration.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **LR Rotation**

Step 2: Right rotation : Rotate right around 30:





The tree is balanced.

LR rule : LR → Left Rotation followed by Right Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Now consider:

10, 30, 20

The BST becomes:

```
   10
    \
     30
    /
   20
```

The path is:

10 → 30 → 20

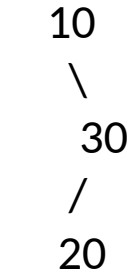
The first move is **right**, followed by **left**.

Therefore, this is an **RL case**. (requires double rotation)

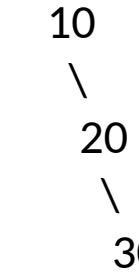
© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Step 1: Right rotation : Rotate right around 30:



becomes:

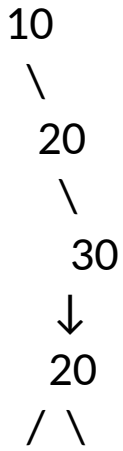


Now we have an RR configuration.

© Confidential Information of PES University. All Rights Reserved - 2026

AVL tree: **RL Rotation**

Step 2: Left rotation : Rotate left around 10:



The tree is balanced.

RL rule : RL → Right Rotation followed by Left Rotation

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree

Splay Tree:

- A **Splay Tree** is a **self-adjusting Binary Search Tree (BST)**
- **Whenever a node is accessed, move that node to the root**
- This operation is called **splaying**

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: salient features

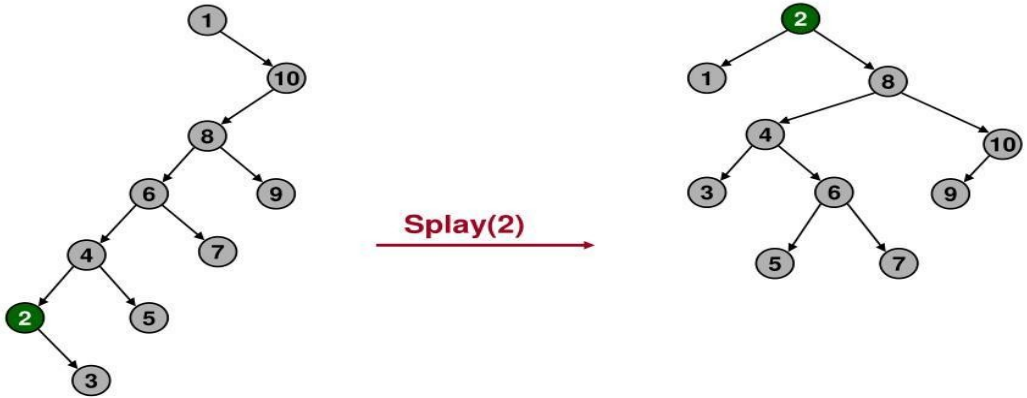
- **No Extra Metadata:** They require **no balance accounting data** (like color bits in red-black trees or height numbers in AVL trees), saving memory.
- **Locality of Reference:** If we access a piece of data once, it moves to the top, making **subsequent lookups of the same data instantaneous**.
- **Efficiency:** While a single worst-case operation can take time proportional to the number of elements, a sequence of m operations is guaranteed to run in time proportional to m * log(n, 2).
- **Dynamic Adaptation:** The tree structure continuously morphs to fit the **actual access pattern** of the workload rather than enforcing a rigid shape.

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree

Splay Example

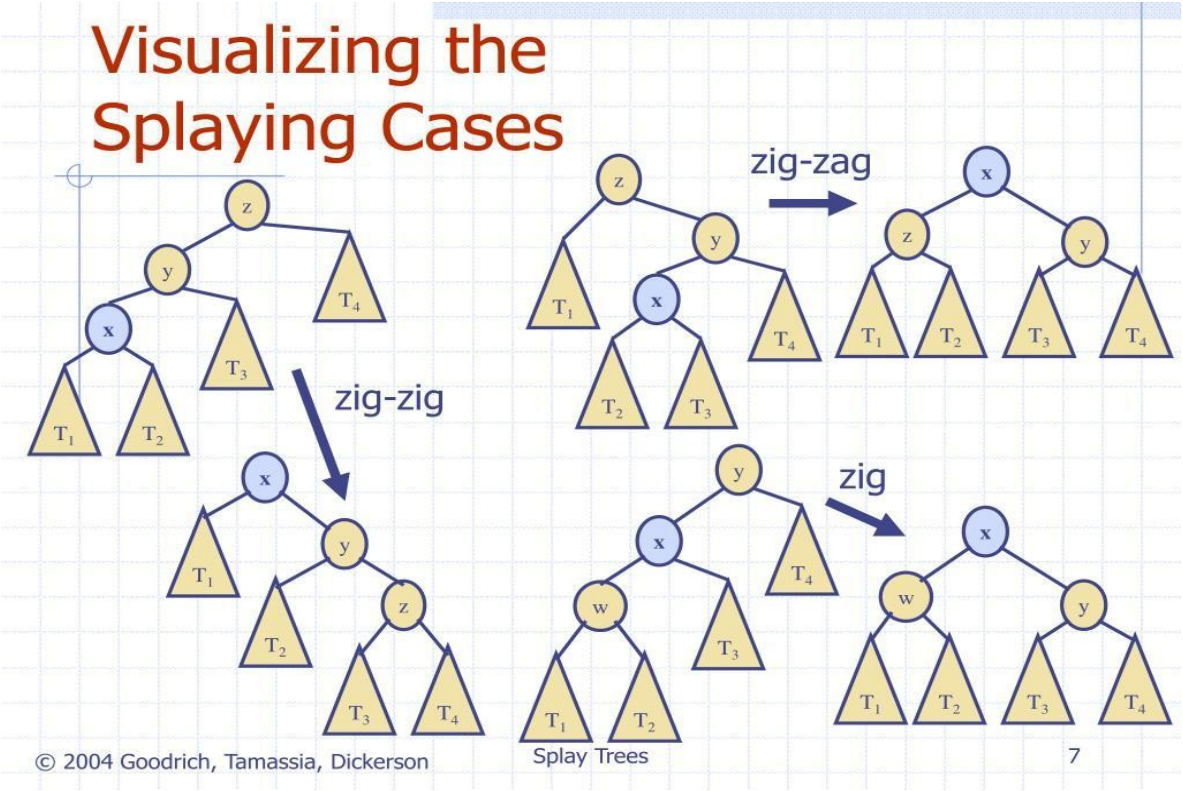
Apply Splay(2, S) to tree S:



28

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree



© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: operations

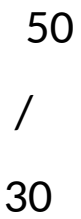
- **Zig**
- **Zig-Zig**
- **Zig-Zag**

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG

- Node accessed is the child of the root

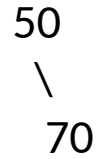
Suppose we access 30:



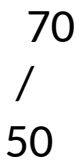
Perform a **right rotation**:



Suppose we access 70



perform a **left rotation:**



Zig rule

Node is child of root

↓

Perform one rotation

↓

Node becomes root

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZIG (LL case)

- node and its parent are both on the **same side**

```
      50
     /
    30
   /
  20
```

We want to access 20.

Here:

20 is the **left child** of 30

30 is the **left child** of 50

So this is **Left-Left (LL)**.

First rotation

Rotate 50 right:

```
      30
     /  \
    20   50
```

Second rotation

Rotate 30 right:

```
    20
      \
       30
         \
          50
```

Zig-ZIG rule

LL => two right rotations

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZIG (RR case)

- node and its parent are both on the **same side**

```
20
  \
   30
    \
     50
```

Access 50.

50 is the **right child** of 30

30 is the **right child** of 20

So this is RR case

First rotation

Rotate 30 left:

```
    30
   /  \
  20   50
```

Second rotation

Rotate 50 left:

```
    50
   /
  30
 /
20
```

Zig-ZIG rule

LL => two right rotations

RR => two left rotations

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZAG

- node and its parent are both on the **opposite side**

Consider:

```
    50
   /
  20
   \
    30
```

30 is the **right child** of 20

20 is the **left child** of 50

Therefore:

Left-Right (LR) configuration.

First rotation

Rotate 20 left:

```
    50
   /
  30
 /
20
```

Second rotation

Rotate 50 right:

```
    30
   / \
  20  50
```

So 30 becomes the root.

Zig-Zag rule

LR => one left; one right

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Finding Running Median using Max-heap and Min-heap

- Step 1:
- Understand Callbacks
  - Parameter of a function makes data flexible
  - Parameter of type pointer to function makes operations flexible
  - Experiment with function programming concept
  - Write functions for
    - Map
    - Filter
    - Reduce

© Confidential Information of PES University. All Rights Reserved - 2026

Finding Running Median using Max-heap and Min-heap

- Step 2:
- Implement a flexible heap
- While initializing provide a function which decides how the comparison is made
- Implement the same heap as
  - MaxHeap
  - MinHeap

© Confidential Information of PES University. All Rights Reserved - 2026

Finding Running Median using Max-heap and Min-heap

- Step 3:
- Find running median
- Data structures:
- Max-Heap (left_half): Stores the smaller half of the numbers.
  - The root gives the largest element of this lower half.
- Min-Heap (right_half): Stores the larger half of the numbers.
  - The root gives the smallest element of this upper half.

© Confidential Information of PES University. All Rights Reserved - 2026

Finding Running Median using Max-heap and Min-heap

- Step 3 (continued):
- Find running median
- Algorithm:
- Step-by-Step Algorithm for Each Incoming Number x
- 1. Insert:
- If left_half is empty or x <= root(left_half),
- push x into left_half.
- Otherwise, push x into right_half.

© Confidential Information of PES University. All Rights Reserved - 2026

Finding Running Median using Max-heap and Min-heap

- Step 3 (continued):
- Find running median
- Algorithm:
- Step-by-Step Algorithm for Each Incoming Number x
- 2. Balance:
- If the size of left_half exceeds right_half by more than 1
- pop the root from left_half and push it into right_half.
- If the size of right_half exceeds left_half,
- pop the root from right_half and push it into left_half.

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Finding Running Median using Max-heap and Min-heap

- Step 3 (continued):
- Find running median
- Algorithm:
- Step-by-Step Algorithm for Each Incoming Number x
- 3. Query Median:
- If total elements N is even: The median is the average of the
- two root elements:
- If total elements N is odd: The median is the root of the heap
- that has the extra element.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Graph

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

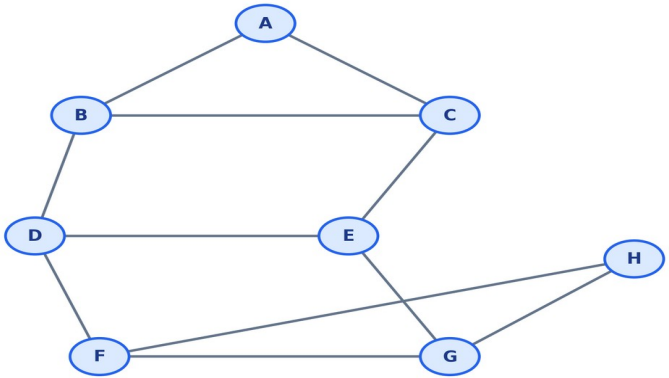
Graph

Definition

A **graph** is a **non-linear data structure** used to represent relationships between objects. It consists of vertices (nodes) and edges connecting pairs of vertices.

G = (V, E)

V is the set of vertices and E is the set of edges.



There are 8 vertices or nodes.

V = { A, B, C, D, E, F, G, H}

There are 11 edges.

AB, AC, BC, BD, CE, DE, DF, FG, EG, GH, FH

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Terminology

Vertex (node): An object represented in the graph, such as A.
Edge: A connection between two vertices, such as AB.
Adjacent vertices: Vertices connected directly by an edge; A and B are adjacent.
Incident edge: An edge is incident on each endpoint; AB is incident on A and B.
Degree: The number of edges incident on a vertex in an undirected graph.
In-degree: The number of directed edges entering a vertex.
Out-degree: The number of directed edges leaving a vertex.
Path: A sequence of vertices joined by edges, such as A–B–D–F–H.
Path length: The number of edges in a path; A–B–D–F–H has length 4.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Terminology

Cycle: A path returning to its starting vertex without repeating intermediate vertices, such as A–B–C–A.
Connected graph: An undirected graph where every vertex is reachable from every other vertex.
Connected component: A maximal connected part of a graph.
Simple graph: A graph without self-loops or multiple edges between the same pair of vertices.
Self-loop: An edge connecting a vertex to itself.
Complete graph: A simple undirected graph where every pair of distinct vertices is connected.
Subgraph: A graph formed from a subset of another graph's vertices and edges.
Cyclic / acyclic graph: A graph that contains / does not contain a cycle.

© Confidential Information of PES University. All Rights Reserved - 2026

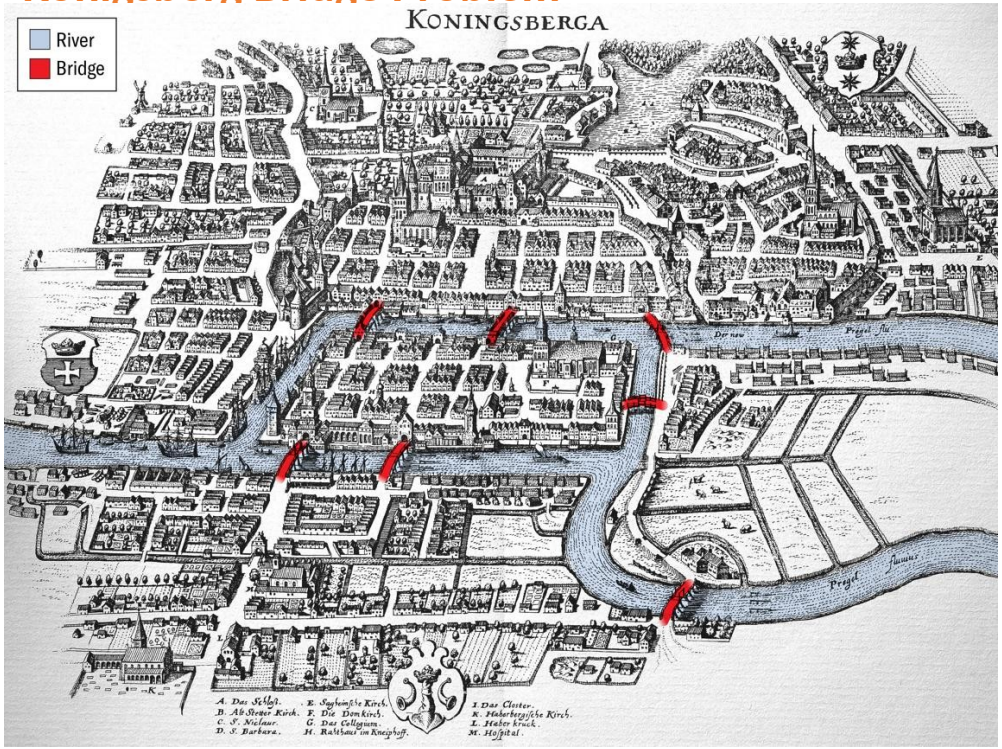
example

Number of vertices: |V| = 8.
Number of edges: |E| = 11.
Neighbours of A: B and C.
Degree of A: deg(A) = 2.
Degree of B: deg(B) = 3 (edges AB, BC and BD).
One path from A to H is A–B–D–F–H; its length is 4.
A cycle is A–B–C–A.
The graph is connected because every vertex can be reached from every other vertex.

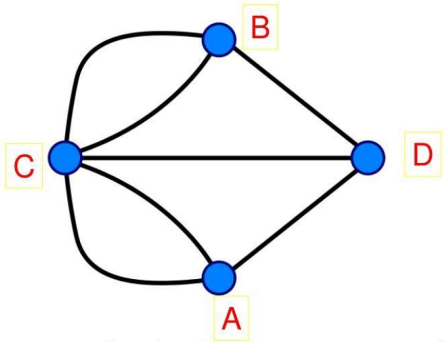
© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Königsberg Bridge Problem



The seven bridges of Konigsberg =



Can you start and end at the same vertex, traversing every edge only once?

© Confidential Information of PES University. All Rights Reserved - 2026

Euler graph and Hamilton graph

A graph is said to be Eulerian if it supports the traversal of all edges once and only once and come back to the starting point. The necessary and sufficient condition is that the degree of each node should be even.

A graph is said to be Hamiltonian if it supports the traversal all vertices once and only once and come back the starting point. We do not have an efficient solution for this.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Applications:

Road and transport networks: Vertices are cities or intersections; edges are roads. Weights can represent distance or travel time.
Social networks: Vertices are people; edges are friendships, follows or interactions.
Computer networks: Vertices are computers or routers; edges are communication links.
Course prerequisites: A directed edge can mean one course must be completed before another.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Applications:

Software dependencies: Vertices are modules, files or tasks; edges represent dependencies.
Airline and logistics networks: Vertices are airports or warehouses; edges represent routes.
Web pages: Vertices are pages and directed edges represent hyperlinks.
Project scheduling: Vertices represent tasks and edges represent precedence constraints

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

ADT:

Operation	Purpose
create_graph()	Create an empty graph.
add_vertex(v)	Insert a vertex.
remove_vertex(v)	Remove a vertex and its incident edges.
add_edge(u, v)	Add an edge between u and v.
remove_edge(u, v)	Remove an edge.
has_vertex(v)	Check whether v exists.
has_edge(u, v)	Check whether an edge exists.
get_neighbors(v)	Return vertices adjacent to v.
degree(v)	Return the degree of v in an undirected graph.
display_graph()	Display vertices and edges.
destroy_graph()	Release resources used by the graph.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)